Git & GitLab Professional – Tag 1

Handout: Git lokal und im Team · Versionsverwaltung · Commits · Fehler korrigieren · Branches · Merge & Rebase · Remote

1 Versionsverwaltung & die drei Bereiche von Git

Versionsverwaltung speichert jede Änderung mit *wer, was, wann, warum*. **Git** ist das Werkzeug auf Ihrem Rechner, **GitLab** die Plattform, auf der das Team sich trifft. Git ist **verteilt**: Jede Kopie enthält die komplette Historie.

Arbeitsverzeichnis → **Staging** → **Repository** → **Remote**

- **Arbeitsverzeichnis:** Ihre Dateien, wie Sie sie bearbeiten – die Werkbank
- **Staging Area:** was in den nächsten Commit soll – der offene Karton
- **Repository (`.git`):** alle gespeicherten Commits
- **Remote:** die gemeinsame Kopie, z. B. auf gitlab.com

`git add` packt in den Karton · `git commit` klebt ihn zu · `git push` bringt ihn zur Post

Was ist ein Commit?

- **Snapshot** aller Dateien zu einem Zeitpunkt
- **Hash** (z. B. `a1b2c3d`): eindeutiger Fingerabdruck
- **Metadaten:** Autor, Zeitpunkt, Nachricht
- **Eltern-Commit:** jeder Commit kennt seinen Vorgänger → Kette

Ein Commit entsteht immer in zwei Schritten:

```
git add src/calc.py  # 1. in den Karton
git commit -m "feat: divide()" # 2. zukleben
```

Einrichten & Hilfe

```
git config --global user.name "Max Muster"
git config --global user.email "max@firma.de"
git config --list --show-origin
git status   # Ihr Navi
git help commit  # Handbuch
git commit -h    # Kurzhilfe
```

2 Repository anlegen & .gitignore

```
git init   # Ordner wird Repository
# vorhandenes Repo holen – per SSH oder HTTPS:
git clone git@gitlab.com:team/app.git
git clone https://gitlab.com/team/app.git
```

- `git init` legt den versteckten Ordner `.git` an – dort lebt die Historie
- `git clone` kopiert alles und merkt sich die Herkunft als `origin`
- Im Alltag klonen Sie in 95 % der Fälle

.gitignore: Muster für Dateien, die nie ins Repository gehören.

```
__pycache__/    *.pyc       .pytest_cache/
.ruff_cache/    .venv/      .env
report.xml      .cache/
```

Secrets (Passwörter, Tokens, Schlüssel) gehören nie ins Repo – sondern in Umgebungs- oder CI/CD-Variablen. Einmal committet, bleibt ein Secret in der Historie → **sofort austauschen**, nicht nur löschen.

Tipp: `git status -u` zeigt auch einzelne Dateien in neuen Unterordnern.

3 Der tägliche Kreislauf

1 · git pull → **2 · bearbeiten** → **3 · add + commit** → **4 · git push**

```
git pull                    # 1
git status -sb              # 2
git add src/calc.py         # 3
git commit -m "feat: add()" # 3
git push                    # 4
```

Ohne Remote (eigenes Projekt) entfallen Schritt 1 und 4. `git add -p` fragt Stück für Stück, was in den Commit soll.

Gute Commit-Nachrichten

schlecht	gut
fix	`fix: Division durch 0 abfangen (Closes #3)`
update	`docs: README um Setup ergänzt`
asdf	`feat(api): Endpunkt /divide ergänzt`

- **Ein Commit = eine logische Änderung**
- Conventional Commits: `feat` · `fix` · `docs` · `refactor` · `test` · `chore` · `ci`
- `Closes #3` schließt in GitLab automatisch Issue 3

4 Historie auswerten: log, diff, blame, bisect

```
git lg   # = log --oneline --graph --all
git log -p -- src/calc.py
git log --author="Max" --since="2 weeks ago"
git show HEAD~2
git log -S "passwort"  # Text gesucht
git blame src/calc.py  # wer, welche Zeile
```

Befehl	vergleicht
`git diff`	Arbeitsverzeichnis ↔ Staging
`git diff --staged`	Staging ↔ letzter Commit
`git diff main..feature` advanced	zwei Branches
`git diff v1.0 v1.1 --stat` advanced	zwei Releases (nur Statistik)

Tipp: Vor jedem Commit einmal `git diff --staged` – Ihr persönliches Mini-Review.

Fehler-Commit finden mit bisect

Wie Zahlenraten: Git halbiert die Historie – 1000 Commits brauchen höchstens 10 Schritte. Bisect braucht **zwei Startpunkte**: einen schlechten und einen guten Commit.

```
git bisect start
git bisect bad     # hier ist der Fehler
git bisect good v1.0  # hier war alles ok
# Git springt in die Mitte, Sie prüfen:
git bisect good  # oder   git bisect bad
# … bis: „<sha> is the first bad commit“
git bisect reset   # fertig
```

Vollautomatisch: Ein Befehl urteilt (Exit-Code 0 = gut, sonst schlecht):

```
git bisect start HEAD v1.0
git bisect run python -m pytest
git bisect reset
```

`bisect run` funktioniert nur nach `bisect start` mit gut und schlecht – sonst gibt es keine Ausgabe.

5 Fehler korrigieren

Die Leitfrage: Ist es schon gepusht? Nein → Sie dürfen die Geschichte umschreiben (`restore`, `commit --amend`, `reset`). Ja → nur eine Korrektur hinzufügen (`revert`).

Beispiel	wirkt auf	wann nutzen?	nach Push?
`git restore calc.py`	Datei	Änderung verwerfen	✔ ok
`git restore --staged calc.py`	Staging	`git add` rückgängig, Änderung bleibt	✔ ok
`git commit --amend -m "Test Nachricht"`	letzter Commit	Nachricht korrigieren oder vergessene Datei ergänzen	✘ nein
`git reset --soft HEAD~1`	Branch-Zeiger	Commit auflösen, Änderungen bleiben gestaged (z. B. Commits zusammenfassen)	✘ nein
`git reset --mixed HEAD~1`	Zeiger + Staging	Commit auflösen, Änderungen neu sortieren (Standard ohne Option)	✘ nein
`git reset --hard HEAD~1`	Zeiger + Staging + Dateien	alles wegwerfen – Vorsicht!	✘ nein
`git revert a1b2c3d`	neuer Gegen-Commit	gepushten Fehler korrigieren – wie eine Stornobuchung	✔ ok

reset als Zeitmaschine: `--soft` Gepäck bleibt im Karton · `--mixed` liegt auf der Werkbank · `--hard` landet im Müll. `HEAD~1` = ein Commit zurück. Faustregel: lokal `reset`, gepusht `revert`.

6 Aliases, Tags & Versionsnummern

Alias	steht für
`git st`	`git status -sb`
`git lg`	`git log --oneline --graph --decorate --all`
`git unstage`	`git restore --staged`
`git amend`	`git commit --amend --no-edit`

```
git config --global alias.st "status -sb"
```

Merksatz: Alias = Kurzwahl (benennt einen Befehl) · Tag = Gedenktafel (benennt einen Commit, bewegt sich nie) · Branch = Lesezeichen, das mitwandert.

```
git tag -a v1.0.0 -m "Release 1.0.0"
git tag -l "v1.*"        # auflisten
git push origin v1.0.0   # einzeln pushen!
```

Semantic Versioning `MAJOR.MINOR.PATCH`:

- **PATCH** 1.4.2 → 1.4.3: Fehlerbehebung
- **MINOR** 1.4.3 → 1.5.0: neue Funktion, alles Alte geht weiter
- **MAJOR** 1.5.0 → 2.0.0: bricht Bestehendes

7 Branches & HEAD

Ein **Branch** ist eine eigene Entwicklungslinie – technisch nur ein beweglicher Name, der auf einen Commit zeigt. Neuer Commit → der Name wandert mit. So bleiben Features und Experimente von `main` getrennt.

```
git switch -c feature/42-divide  # neu
git branch -vv     # alle Branches
git switch main    # zurück
git branch -d feature/42-divide  # löschen
```

Namensschema: `feature/`, `fix/`, `chore/` + Issue-Nummer + Stichwort. Ältere Anleitungen: `git checkout -b` = dasselbe.

HEAD ist der „Sie sind hier"-Punkt: Normalerweise zeigt HEAD auf einen Branch und der Branch auf einen Commit.

- `git switch feature` bewegt HEAD auf einen anderen Branch
- `HEAD~1`, `HEAD~2`: ein bzw. zwei Commits zurück
- **Detached HEAD:** HEAD zeigt direkt auf einen Commit (z. B. nach `bisect`). Neue Commits dort gehören zu keinem Branch → sichern mit `git switch -c rettung`

8 Merge, Konflikte & Rebase

Fast-Forward

`main` hat sich nicht bewegt → Git schiebt den Zeiger einfach vor, kein neuer Commit.

```
git switch main
git merge feature/divide  # „Fast-forward“
```

3-Way-Merge

Beide Linien haben neue Commits → Git erzeugt einen **Merge-Commit** mit zwei Eltern. GitLab macht das beim Merge Request standardmäßig. Erzwingen: `git merge --no-ff`.

Konflikt

Zwei Linien ändern **dieselbe Zeile** unterschiedlich. Git trifft keine fachliche Entscheidung für Sie:

```
<<<<<<< HEAD  ← Ihre Version
timeout = 60
=======
timeout = 10
>>>>>>> feature/perf  ← die hereinkommende Version
```

Konflikt lösen in 5 Schritten

1. **Erkennen:** `git status` zeigt „both modified“
2. **Öffnen:** Datei im Editor öffnen
3. **Entscheiden:** richtige Fassung bauen (z. B. `timeout = 30`)
4. **Markieren:** alle Marker löschen, `git add datei`
5. **Abschließen:** `git commit` bzw. `git rebase --continue`

Notbremse: `git merge --abort` / `git rebase --abort`. Beim Rebase steht oben (HEAD) die Basis, unten Ihr Commit – umgekehrt wie beim Merge.

Rebase

Setzt Ihre Commits auf eine neue Basis → gerade Linie, aber **neue Hashes**.

```
git switch feature && git rebase main
git push --force-with-lease  # nur eigener Branch
```

	Merge	Rebase
Historie	ehrlich, mit Verzweigungen	gerade Linie
Hashes	bleiben	neu
Einsatz	Integration in main (MR)	eigenen Branch aktuell halten

Goldene Regel: Niemals Commits rebasen, die andere schon haben.

9 Remote: im Team arbeiten

- **Remote:** das Repository auf dem Server (gitlab.com)
- **origin:** Standardname des Remotes nach `git clone`
- **origin/main:** Lesezeichen „so sah main beim letzten Abgleich aus“
- **Upstream:** der Remote-Branch, mit dem Ihr Branch verknüpft ist

```
git remote -v       # Remotes anzeigen
git fetch --prune   # Neues holen
git pull            # holen + einbauen
git push            # hochladen
```

- **fetch:** ungefährlich – ändert Ihre Arbeit nicht
- **pull:** mit `pull.rebase=true` landen Ihre Commits sauber obendrauf
- **push:** klappt nur mit dem neuesten Stand

Mit Branches der anderen arbeiten

```
git push -u origin feature/max-idee # hoch
git branch -r                # alle auf dem Server
git log origin/feature/lea-idee  # nur ansehen
git switch feature/lea-idee  # mitarbeiten
git push origin --delete feature/max-idee
```

Push abgelehnt? Kein Drama!

```
! [rejected]   main -> main (fetch first)
hint: Updates were rejected because the remote
      contains work that you do not have locally.
```

Auf dem Server liegt Arbeit, die Sie noch nicht haben – Git schützt sie vor dem Überschreiben.

git pull → **ggf. Konflikt lösen** → **git push**

Niemals `--force` auf geteilten Branches – höchstens `--force-with-lease` auf dem eigenen.

10 Team-Workflows

Workflow	Idee	passt, wenn …
GitHub Flow	main + kurze Feature-Branches + Merge Request	Sie oft und kontinuierlich ausliefern
GitLab Flow	zusätzlich Branches pro Umgebung (staging, production)	Sie Test-, Staging- und Produktionsumgebung haben
Trunk-Based	alle integrieren täglich in main, Feature Flags	Ihr Team sehr gute Tests hat
Git Flow	develop-, release- und hotfix-Branches	Kunden mehrere Versionen parallel nutzen

Für die meisten Teams: **GitHub oder GitLab Flow** – kurze Branches, geschützter main, Review.

11 Ihre Übungen heute

Übung	Arbeitsform	Inhalt
1 · Projektstart	selbst	Konfiguration, `init`, `.gitignore`, zwei saubere Commits
2 · Detektivarbeit	Gruppe (Breakout)	`log -S`, `bisect`, `revert`, `restore`
3 · Branches & Konflikte	selbst + zu zweit	Feature-Branch, Fast-Forward, Tag `v0.1.0`, Konflikt per Merge und Rebase
4 · Team-Sync	gemeinsam	clone, abgelehnter Push, echter Konflikt, Remote Branches

Hängen geblieben? Erst `git status` und `git lg` – Git schlägt meist den nächsten Schritt vor. Anleitungen: `$SEMINAR/tag-1/` · Spickzettel: `$SEMINAR/nachschlagen/`